

EKaNa – Experience Karnataka Naturally

Full Developer Blueprint

Stack Choices (confirmed): Next.js (frontend), FastAPI (backend), PostgreSQL (database)

Prepared for: Project Team

Prepared by: Pradhyumna (Project Lead)

1. Executive Summary

This document is a comprehensive developer blueprint for the EKaNa platform — a tourism web application showcasing Karnataka's destinations, communities, and sustainable experiences. It provides folder structures, API contracts, database schema, component designs, environment variables, CI/CD guidance, deployment instructions, and a detailed task breakdown.

2. System Architecture

High-level components:

- Next.js frontend (SSG/SSR on Vercel) - UI, routing, SEO, client-side state
- FastAPI backend (Render/Railway) - REST endpoints, authentication, business logic
- PostgreSQL database (Supabase/Neon) - persistent storage for destinations, users, community data
- Cloudinary for image hosting; CDN via Cloudflare
- Optional: Redis for caching; Celery/RQ for background jobs

3. Frontend Folder Structure (Next.js)

```
/frontend
├── public/                # static assets (images, icons)
├── src/
│   ├── components/       # reusable UI components
│   │   ├── common/      # Navbar, Footer, Button, Modal
│   │   └── domain/      # DestinationCard, MapView, Itinerary
│   ├── pages/            # Next.js pages (index.js, discover, destination/[id].js)
│   ├── hooks/            # custom hooks (useFetch, useAuth, useMap)
│   ├── contexts/         # React context / Zustand stores
│   ├── services/         # API wrappers (destinations, auth, community)
│   ├── styles/           # tailwind.css, global.css
│   └── utils/            # helpers, constants
├── next.config.js
└── package.json
```

4. Backend Folder Structure (FastAPI)

```
/backend
├── app/                  # FastAPI app
│   ├── main.py
│   ├── api/
│   │   ├── v1/
│   │   │   ├── routes/  # destinations.py, users.py, community.py
│   │   │   └── deps.py
│   ├── core/            # config, security utils
│   ├── models/          # SQLAlchemy models
│   ├── schemas/         # Pydantic schemas
│   ├── crud/            # DB operations
│   ├── services/        # email, image upload, background tasks
│   ├── db.py            # DB connection
│   └── alembic/         # migrations
```

■■■ requirements.txt

5. API Endpoints & Contracts (Selected)

Destinations

Method	Endpoint	Request Body	Response (200)
GET	/api/v1/destinations	Query params: category, district, page, limit, sort	List of destination objects (paginated)
GET	/api/v1/destinations/{id}	-	Destination object with details
POST	/api/v1/destinations	Auth Admin. JSON: {name, category, district, description, image_urls, latitude, longitude, best_time, tags}	Created destination object

Users & Auth

Method	Endpoint	Request Body	Response (200)
POST	/api/v1/auth/register	JSON: {name, email, password}	User object + JWT
POST	/api/v1/auth/login	JSON: {email, password}	JWT token
GET	/api/v1/auth/me	Auth: Bearer token	User profile

Community Listings

Method	Endpoint	Request Body	Response (200)
GET	/api/v1/community	Query params: type, district, page	List of community items
POST	/api/v1/community	JSON/form: {name, type, contact, district, description, image_urls}	Created listing (pending approval)

6. Database Schema (PostgreSQL) — Key Tables & SQL

Destinations table (example SQL):

```
CREATE TABLE destinations (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  category VARCHAR(50) NOT NULL,  
  district VARCHAR(100),  
  description TEXT,  
  image_urls TEXT[],  
  latitude NUMERIC(10,6),  
  longitude NUMERIC(10,6),  
  best_time VARCHAR(100),  
  tags TEXT[],  
  created_at TIMESTAMP DEFAULT now(),  
  updated_at TIMESTAMP DEFAULT now()  
);
```

Community table (example SQL):

```
CREATE TABLE community (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  type VARCHAR(50),  
  district VARCHAR(100),  
  contact VARCHAR(255),  
  description TEXT,  
  image_urls TEXT[],  
  created_at TIMESTAMP DEFAULT now()  
);
```

7. Environment Variables (.env) — Suggested Keys

Key	Example Value
-----	---------------

NEXT_PUBLIC_API_URL	https://api.ekana.in
NEXT_PUBLIC_CLOUDINARY_URL	https://res.cloudinary.com/<your-cloud>/
DATABASE_URL	postgresql://user:pass@host:port/dbname
SECRET_KEY	supersecretkey
JWT_SECRET	jwtsecret
CLOUDINARY_API_KEY	
CLOUDINARY_API_SECRET	
SENTRY_DSN	optional for error monitoring

8. Frontend – Key Code Snippets

API wrapper (axios):

```
import axios from 'axios';
const API = axios.create({ baseURL: process.env.NEXT_PUBLIC_API_URL });

export const getDestinations = (params) => API.get('/api/v1/destinations', { params });
export const getDestination = (id) => API.get(`/api/v1/destinations/${id}`);
```

Destination card component (simplified):

```
export default function DestinationCard({ d }) {
  return (
    <a href={`destination/${d.id}`} className="block rounded-lg overflow-hidden shadow">
      <img src={d.image_urls[0]} alt={d.name} className="w-full h-48 object-cover" />
      <div className="p-3">
        <h3 className="text-lg font-semibold">{d.name}</h3>
        <p className="text-sm text-gray-600">{d.district}</p>
      </div>
    </a>
  );
}
```

9. Authentication & Authorization

Use JWT tokens issued by FastAPI on login. Refresh tokens optional. Protect admin routes on backend with role checks.

Frontend: store token in httpOnly cookie (best) or localStorage (less secure). Use Axios interceptor to attach Authorization header.

10. Image Upload Flow

Use Cloudinary signed uploads or direct server uploads to Cloudinary. Store returned image URLs in DB (image_urls array). Use transformations for responsive sizes.

11. Performance: Caching, Pagination & Filtering

- Implement server-side pagination (limit, offset) for listing endpoints.
- Add Redis caching for frequent reads (top destinations, featured lists).
- Use database indexes on category, district, and tags columns.
- Debounce search on frontend.

12. Testing Strategy

- Frontend unit tests: Jest + React Testing Library

- E2E tests: Playwright or Cypress (Discover flow, Destination detail, Community submission)
- Backend tests: Pytest for API endpoints, fixtures for DB
- Integration tests: run in CI using a test database (Postgres docker) and mocked Cloudinary

13. CI/CD and Deployment

- Use GitHub Actions pipeline: run lint, tests, build, and deploy to Vercel/Render.

```
name: CI
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Set up Node
        uses: actions/setup-node@v2
        with: node-version: 18
      - run: npm ci && npm run build --if-present && npm test
```

14. Security & Compliance

- Use HTTPS everywhere (Cloudflare + origin SSL).
- Sanitize and validate all inputs on backend (Pydantic schemas).
- Rate-limit endpoints to avoid abuse (e.g., 100/min for public endpoints).
- Secure environment variables and secrets in deployment platform.

15. Monitoring & Analytics

- Use Sentry for error monitoring.
- Use Google Analytics + Google Search Console for SEO tracking.
- Use simple admin dashboard for traffic and featured destination metrics.

16. Team Tasks & Sprint Plan (8 weeks)

Sprint	Duration	Key Deliverables
Sprint 0	1 week	Project setup, repo, initial infra, DB schema
Sprint 1	2 weeks	Navbar, Home, DestinationCard, sample data
Sprint 2	2 weeks	Discover page, Map integration, Filters
Sprint 3	2 weeks	Destination details, Community flows
Sprint 4	1 week	Testing, SEO, Deployment

Appendix A — Useful SQL & Indexes

```
-- Indexes for performance
CREATE INDEX idx_dest_category ON destinations (category);
CREATE INDEX idx_dest_district ON destinations (district);
CREATE INDEX idx_dest_tags ON destinations USING gin (tags);

-- Paginated query example
SELECT id, name, district, image_urls[1] as hero_image
FROM destinations
WHERE category = 'temple'
ORDER BY created_at DESC
LIMIT 12 OFFSET 0;
```

Contact & Next Steps

Share this document with the team. Next: set up repos and run Sprint 0 tasks (see Sprint table). For any custom deliverable (e.g., mobile app scaffold), request a follow-up document.